

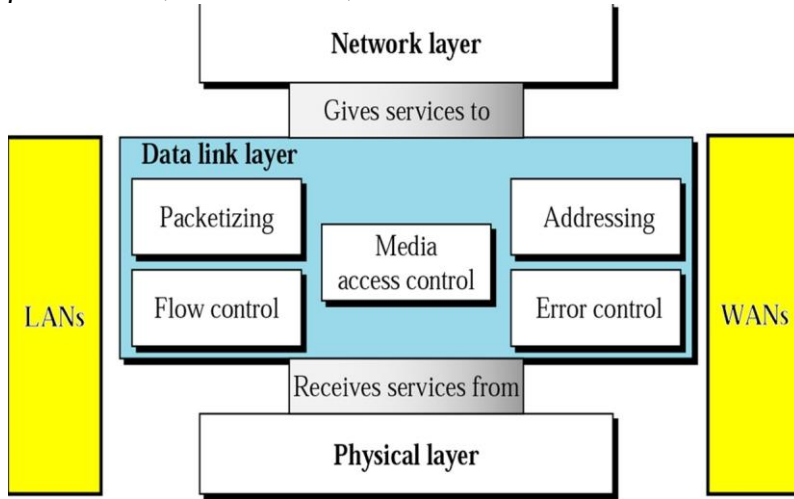
COMPUTER NETWORKS

UNIT – 2

DATA LINK LAYER

DATA LINK LAYER:

The data link layer transforms the physical layer, a raw transmission facility, to a link responsible for node-to-node (hop-to-hop) communication. Specific responsibilities of the data link layer include *framing, addressing, flow control, error control, and media access control*.

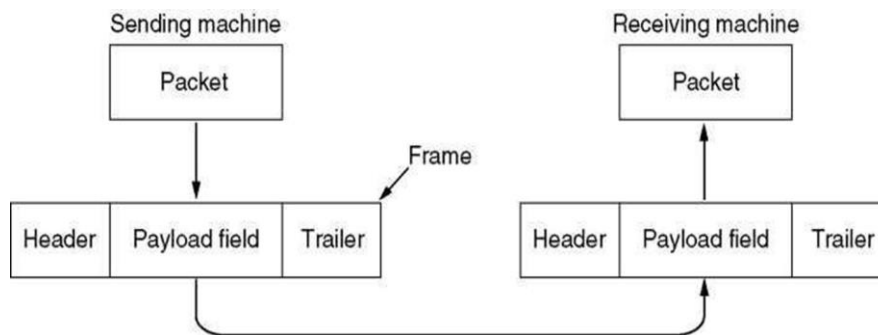


DATA LINK LAYER DESIGN ISSUES

The data link layer uses the services of the physical layer to send and receive bits over communication channels. It has a number of functions, including:

1. Providing a well-defined service interface to the network layer.
2. Dealing with transmission errors.
3. Regulating the flow of data so that slow receivers are not swamped by fast senders.

To accomplish these goals, the data link layer takes the packets it gets from the network layer and encapsulates them into frames for transmission. Each frame contains a frame header, a payload field for holding the packet, and a frame trailer, as illustrated in Fig.



SERVICES PROVIDED TO THE NETWORK LAYER

The function of the data link layer is to provide services to the network layer. The principal service is transferring data from the network layer on the source machine to the network layer on the destination machine. The data link layer can be designed to offer various services. The actual services that are offered vary from protocol to protocol. Three reasonable possibilities that we will consider in turn are:

1. Unacknowledged connectionless service.
2. Acknowledged connectionless service.
3. Acknowledged connection-oriented service.

1. Unacknowledged connectionless service:

Unacknowledged connectionless service consists of having the source machine send independent frames to the destination machine without having the destination machine acknowledge them. Ethernet is a good example of a data link layer that provides this class of service. No logical connection is established beforehand or released afterward. If a frame is lost due to noise on the line, an attempt is made to detect the loss or recover from it in the data link layer. This class of service is appropriate when the error rate is very low, so recovery is left to higher layers.

2. Acknowledged connectionless service:

When this service is offered, there are still no logical connections used, but each frame sent is individually acknowledged. In this way, the sender knows whether a frame has arrived correctly or been lost. If it has not arrived within a specified time interval, it can be sent again. This service is useful over unreliable channels, such as wireless systems. 802.11 (WiFi) is a good example of this class of service.

3. Acknowledged connection-oriented service:

The most sophisticated service the data link layer can provide to the network layer is connection-oriented service. With this service, the source and destination machines establish a connection before any data are transferred. Each frame sent over the connection is numbered, and the data link layer guarantees that each frame sent is indeed received. Furthermore, it guarantees that each frame is received exactly once and that all frames are received in the right order.

When connection-oriented service is used, transfers go through three distinct phases. In the first phase, the connection is established by having both sides initialize variables and counters needed to keep track of which frames have been received and which ones have not. In the second phase, one or more frames are actually transmitted. In the third and final phase, the connection is released, freeing up the variables, buffers, and other resources used to maintain the connection.

FRAMING

To provide service to the network layer, the data link layer must use the service provided to it by the physical layer. What the physical layer does is accept a raw bit stream and attempt to deliver it to the destination. If the channel is noisy, as it is for most wireless and some wired links, the physical layer will add some redundancy to its signals to reduce the bit error rate to a tolerable level. However, the bit stream received by the data link layer is not guaranteed to be error free. Some bits may have different values and the number of bits received may be less than, equal to, or more than the number of bits transmitted. It is up to the data link layer to detect and, if necessary, correct errors.

The data link layer breaks up the bit stream into discrete frames, computes a short token called a checksum for each frame, and includes the checksum in the frame when it is transmitted.

When a frame arrives at the destination, the checksum is recomputed. If the newly computed checksum is different from the one contained in the frame, the data link layer knows that an error has occurred and takes steps to deal with it.

DLL translates the physical layer's raw bit stream into discrete units (messages) called **frames**.

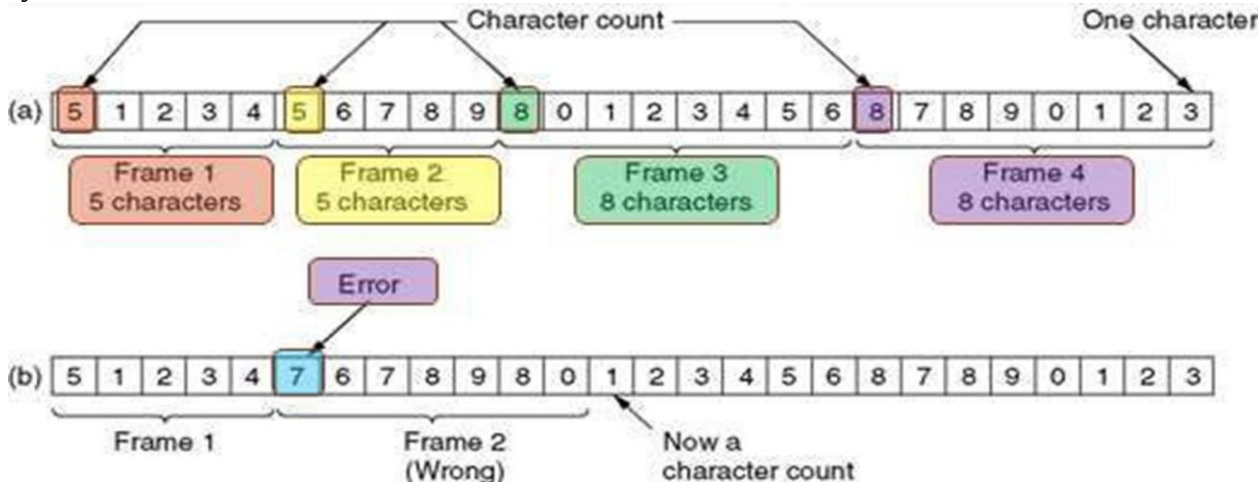
A good design must make it easy for a receiver to find the start of new frames while using little of the channel bandwidth. We will look at **four methods**:

1. **Byte count.**
2. **Flag bytes with byte stuffing.**
3. **Flag bits with bit stuffing.**
4. **Physical layer coding violations.**

1. Byte count (Character Count) :

This framing method uses a field in the header to specify the number of bytes in the frame. When the data link layer at the destination sees the byte count, it knows how many bytes follow and hence where the end of the frame is. This technique is shown in Fig.(a) For four small example frames of sizes 5, 5, 8, and 8 bytes, respectively.

The trouble with this algorithm is that the count can be garbled by a transmission error. For example, if the byte count of 5 in the second frame of Fig.(b) becomes a 7 due to a single bit flip, the destination will get out of synchronization. It will then be unable to locate the correct start of the next frame.

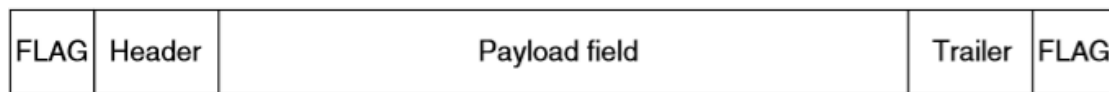


2. Flag bytes with byte stuffing:

This framing method gets around the problem of resynchronization after an error by having each frame start and end with special bytes. Often the same byte, called a **flag byte**, is used as both the starting and ending delimiter. This byte is shown in Fig.(a) as **FLAG**. Two consecutive flag bytes indicate the end of one frame and the start of the next. Thus, if the receiver ever loses synchronization it can just search for two flag bytes to find the end of the current frame and the start of the next frame.

However, there is still a problem we have to solve. It may happen that the flag byte occurs in the data, especially when binary data such as photographs or songs are being transmitted. This situation would interfere with the framing. One way to solve this problem is to have the sender's data link layer insert a special escape byte (ESC) just before each "accidental" flag byte in the data.

The data link layer on the receiving end removes the escape bytes before giving the data to the network layer. This technique is called **byte stuffing**.



(a)



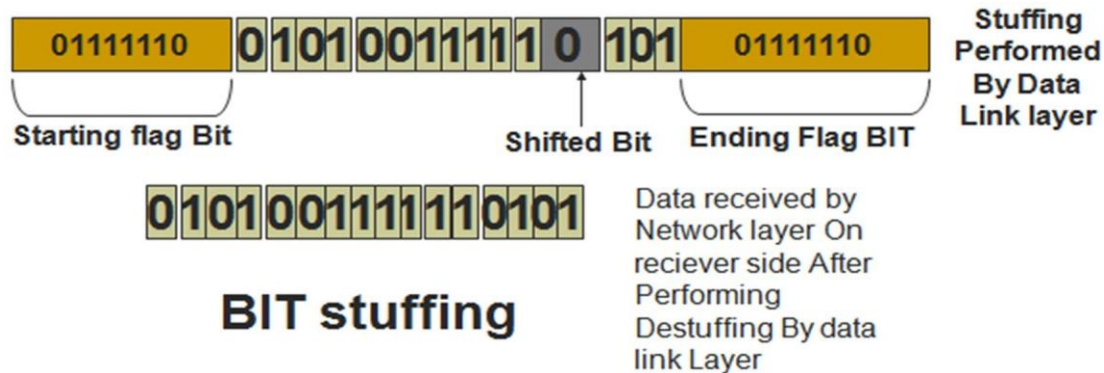
Four examples of byte sequences before and after byte stuffing.

3. Flag bits with bit stuffing:

Framing can be also be done at the bit level, so frames can contain an arbitrary number of bits made up of units of any size. It was developed for the once very popular **HDLC (Highlevel Data Link Control)**

protocol. Each frame begins and ends with a special bit pattern, 01111110 or 0x7E in hexadecimal. This pattern is a flag byte. Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream. This **bit stuffing** is analogous to byte stuffing, in which an escape byte is stuffed into the outgoing character stream before a flag byte in the data. It also ensures a minimum density of transitions that help the physical layer maintain synchronization. USB (Universal Serial Bus) uses bit stuffing for this reason.

When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically destuffs (i.e., deletes) the 0 bit. Just as byte stuffing is completely transparent to the network layer in both computers, so is bit stuffing. If the user data contain the flag pattern, 01111110, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110. Figure gives an example of bit stuffing.



4. Physical layer coding violations:

- This Framing Method is used only in those networks in which Encoding on the Physical Medium contains some redundancy.
- Some LANs encode each bit of data by using two Physical Bits i.e. Manchester coding is used. Here, Bit 1 is encoded into high- low (10) pair and Bit 0 is encoded into low-high (01) pair.
- The scheme means that every data bit has a transition in the middle, making it easy for the receiver to locate the bit boundaries. The combinations high-high and low-low are not used for data but are used for delimiting frames in some protocols.

ERROR CONTROL:

Error control is concerned with ensuring that all frames are eventually delivered (possibly in order) to a destination. How? Three items are required.

- **Acknowledgements:** Typically, reliable delivery is achieved using the “acknowledgments with retransmission” paradigm, whereby the receiver. returns a special acknowledgment (ACK) frame to the sender indicating the correct receipt of a frame. In some systems, the receiver also returns a negative acknowledgment (NACK) for incorrectly-received frames. This is nothing more than a hint to the sender so that it can retransmit a frame right away without waiting for a timer to expire.
- **Timers:** One problem that simple ACK/NACK schemes fail to address is recovering from a frame that is lost? Retransmission timers are used to resend frames that don't produce an ACK. When sending a frame, schedule a timer to expire at some time after the ACK should have been returned. If the timer goes o, retransmit the frame.

- **Sequence Numbers:** Retransmissions introduce the possibility of duplicate frames. To suppress duplicates, add sequence numbers to each frame, so that a receiver can distinguish between new frames and old copies.

FLOW CONTROL:

Flow control deals with controlling the speed of the sender to match that of the receiver.

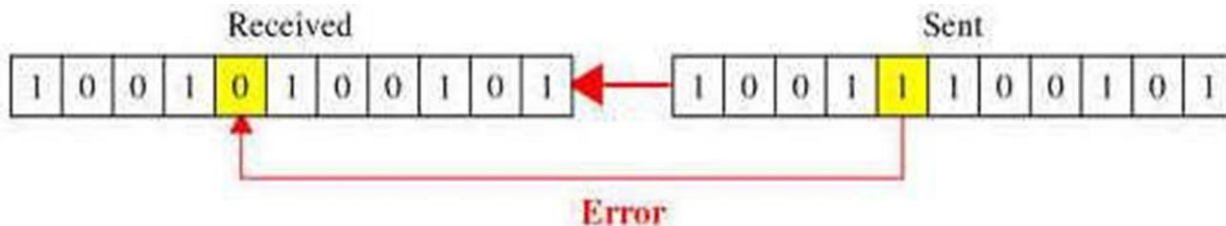
Two Approaches:

- **feedback-based flow control**, the receiver sends back information to the sender giving it permission to send more data or at least telling the sender how the receiver is doing
- **rate-based flow control**, the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

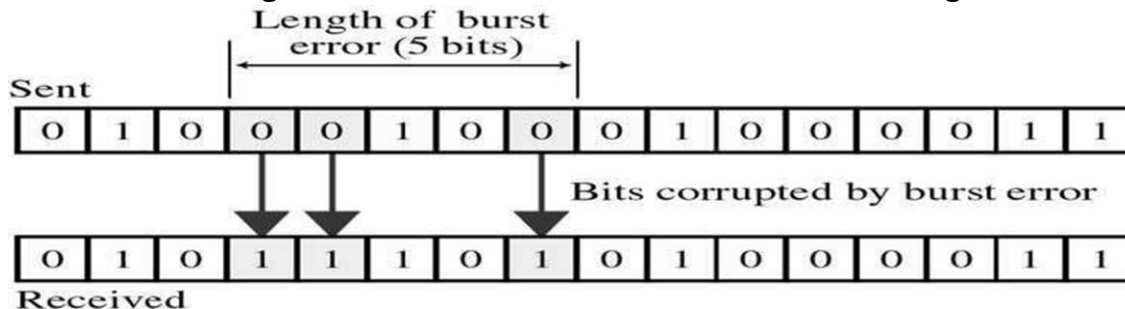
TYPES OF ERRORS:

There are two main types of errors in transmissions:

1. **Single bit error:** It means only one bit of data unit is changed from 1 to 0 or from 0 to 1.



2. **Burst error:** It means two or more bits in data unit are changed from 1 to 0 from 0 to 1. In burst error, it is not necessary that only consecutive bits are changed. The length of burst error is measured from first changed bit to last changed bit

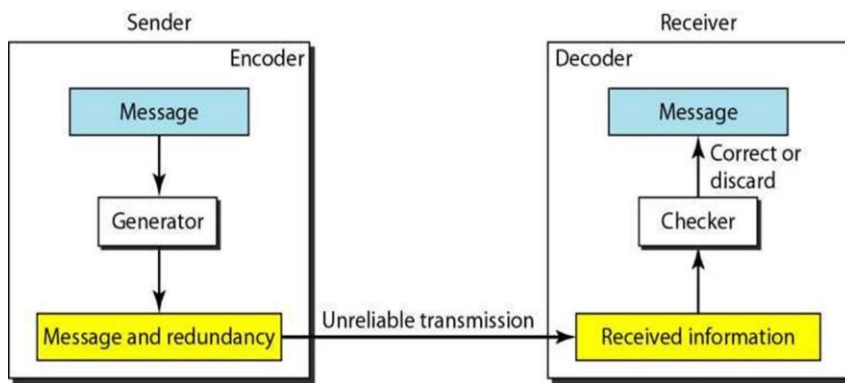


ERROR DETECTION AND CORRECTION:

Network designers have developed two basic strategies for dealing with errors. Both add redundant information to the data that is sent. One strategy is to include enough redundant information to enable the receiver to deduce what the transmitted data must have been. The other is to include only enough redundancy to allow the receiver to deduce that an error has occurred and have it request a retransmission. The former strategy uses **error-correcting codes** and the latter uses **error-detecting codes**.

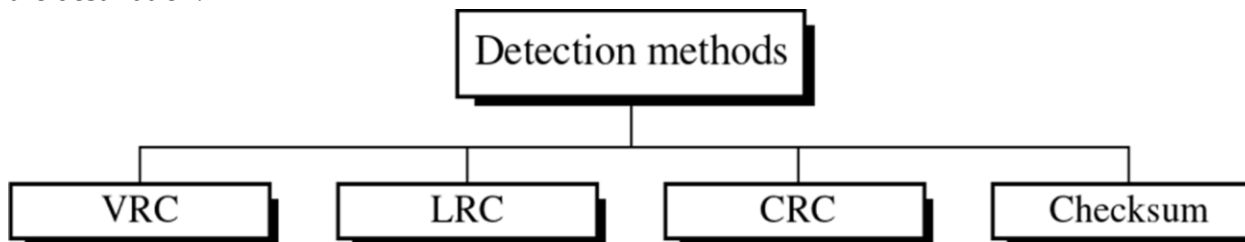
Error Detecting Codes: Include enough redundancy bits to *detect* errors and use ACKs and retransmissions to recover from the errors.

Error Correcting Codes: Include enough redundancy to detect and correct errors.



ERROR-DETECTING CODES:

Error detection means to decide whether the received data is correct or not without having a copy of the original message. Error detection uses the concept of redundancy, which means adding extra bits for detecting errors at the destination.



1. Vertical Redundancy Check(VRC):

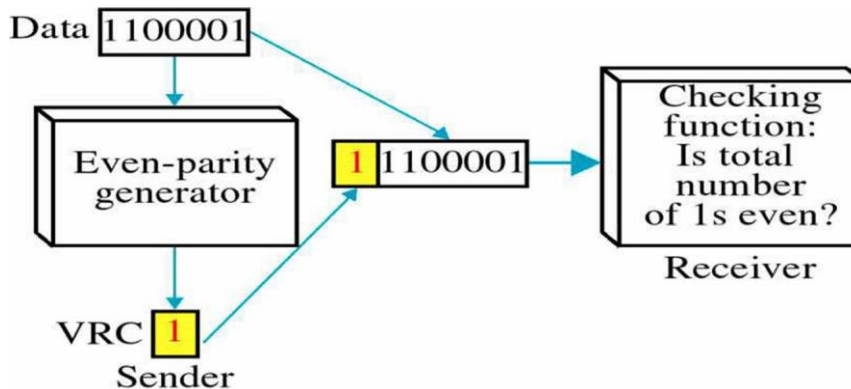
Append a single bit at the end of data block such that the number of one's is even

→ A Even Parity (odd parity is similar) 0110011

→ 01100110

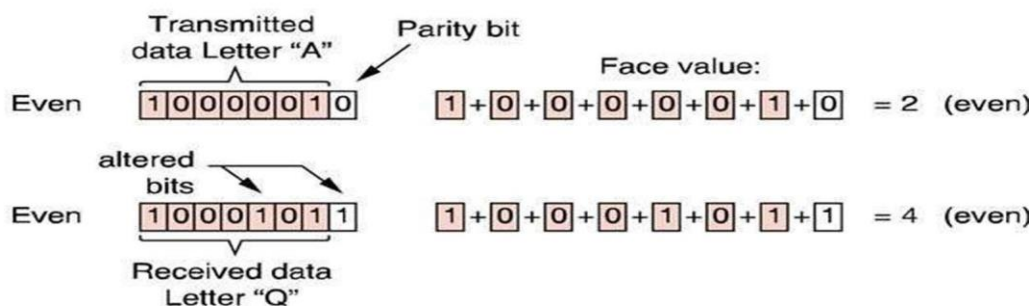
0110001 → 01100011

VRC is also known as **Parity Check**. Detects all odd-number errors in a data block.



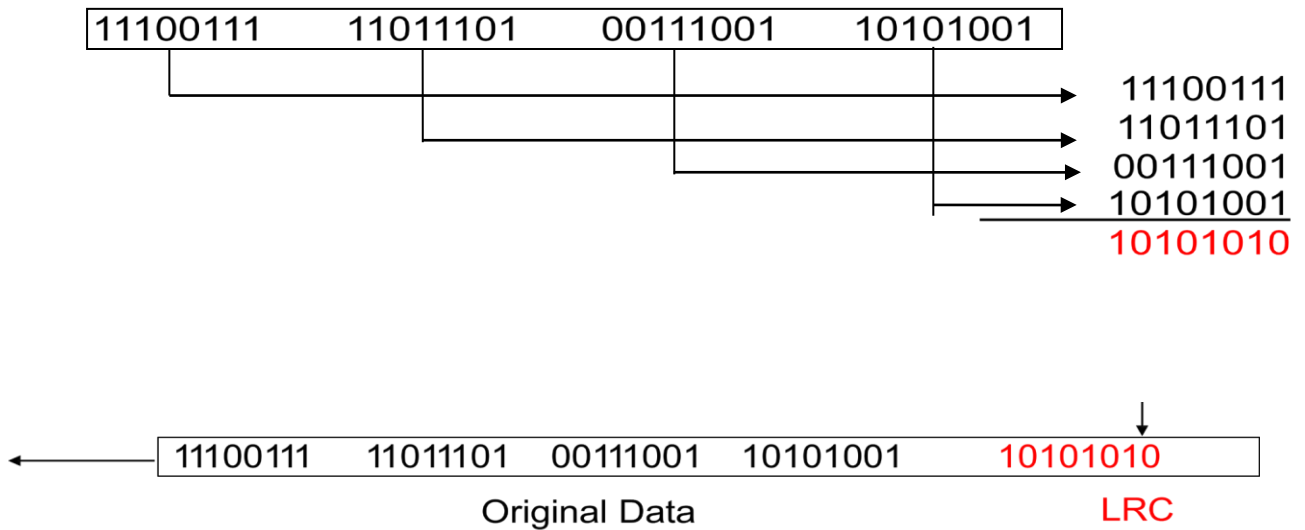
The problem with parity is that it can only detect odd numbers of bit substitution errors, i.e. 1 bit, 3bit, 5, bit, etc. Errors. If there two, four, six, etc. bits which are transmitted in error, using VRC will not be able to detect the error.

Multiple bit errors/even parity



2. Longitudinal Redundancy Check(LRC):

- Longitudinal Redundancy Checks (LRC) seek to overcome the weakness of simple, bit-oriented, one-directional parity checking.
- LRC adds a new character (instead of a bit) called the Block Check Character (BCC) to each block of data. Its determined like parity, but counted longitudinally through the message (also vertically)
- Its has better performance over VRC as it detects 98% of the burst errors (>10 errors) but less capable of detecting single errors



3. Cyclic Redundancy Check(CRC):

The cyclic redundancy check, or CRC, is a technique for detecting errors in digital data, but not for making corrections when errors are detected. The CRC (**Cyclic Redundancy Check**), also known as **apolynomial code**

Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only. For example, 110001 has 6 bits and thus represents a six-term polynomial with coefficients 1, 1, 0, 0, 0, and 1: $1x^5 + 1x^4 + 0x^3 + 0x^2 + 0x^1 + 1x^0$.

Polynomial arithmetic is done modulo 2, according to the rules of algebraic field theory. It does not have carries for addition or borrows for subtraction. Both addition and subtraction are identical to **exclusive OR**. For example:

10011011	00110011	11110000	01010101
+ 11001010	+ 11001101	− 10100110	− 10101111
<u>01010001</u>	<u>11111110</u>	<u>01010110</u>	<u>11111010</u>

When the polynomial code method is employed, the sender and receiver must agree upon a **generator polynomial, $G(x)$** , in advance. Both the high- and low order bits of the generator must be 1. To compute the CRC for some frame with m bits corresponding to the polynomial $M(x)$, the frame must be longer than the generator polynomial. The idea is to append a CRC to the end of the frame in such a way that the polynomial represented by the check summed frame is divisible by $G(x)$. When the receiver gets the check summed frame, it tries dividing it by $G(x)$. If there is a remainder, there has been a transmission error.

CRC Example:

Generator polynomial:

CRC16:11000000000000101

CRCCIP:10001000000100001

CRC12:1100000001111,Data:100100

Sender divides data with Generator polynomial .modulo -2 division is used here with X-OR operation.the remainder is taken as CRC.CRC is appended with data and is sent to receiver.receiver takes the data and divides with generator polynomial.if the remainder is 0 ,the data is received without errors.if the remainder is with 1 the data is received with errors and receiver discards the data.

Checksum:

Checksum is the error detection scheme used in IP, TCP & UDP.

- Here, the data is divided into k segments each of n bits. In the sender's end the segments are added using 1's complement arithmetic to get the sum. The sum is complemented to get the checksum. The checksum segment is sent along with the data segments
- At the receiver's end, all received segments are added using 1's complement arithmetic to get the sum. The sum is complemented. If the result is zero, the received data is accepted; otherwise discarded
- The checksum detects all errors involving an odd number of bits. It also detects most errors involving even number of bits.

Checksum Example:

Data:10110011,10101011,01011010,11010101

Entire data partitioned into 4 parts ,each part consists of 8 bits,perform binary addition on data.the final sum is taken and one's compliment is performed on sum.the result is checksum .the check sum is appended to data and sent to the receiver.

At the receiver binary addition is performed on all parts of data and checksum .the final sum is taken and one's compliment is performed on sum. If the result is all 0s,then data is received without errors ,if there are 1 s in the result then the data is received with errors.

Example:

10110011

10101011

1 11

1,0 1 0 1 1 1 10

1

```

01011111
01011010
1  1 1 1 1

```

```

1 0 1 1 1 0 0 1
11010101

```

```

1,10001110
      1

```

```

Sum- 10001111

```

Checksum:01110000

Receiver side:

```

      10110011
      10101011
      1   11

```

```

1,0 1  0 1 1 1 10
      1

```

```

01011111
01011010
1  1 1 1 1

```

```

1 0 1 1 1 0 0 1
11010101

```

```

1,10001110
      1

```

```

Sum- 10001111

```

After adding check sum to the result

```

10001111
01110000

```

00000000 , since the result is all 0,s the data is received without errors

Hamming code:

Hamming code is used for error detection and correction.

In 7 bit hamming code there are 4 data bits and 3 parity bits. to find the value of parity bits even parity or odd parity is applied on data bits.

Example: Data bits: 1011 , by numbering bits in hamming code: 7 6 5 4 3 2 1

Parity bit position is calculated as follows: parity bit p1 position = $2^0 = 1$

Parity bit position is calculated as follows : parity bit p2 position = $2^1 = 2$

Parity bit position is calculated as follows : parity bit p4 position = $2^2 = 4$

Remaining data bits positions and values are: D3=1, D5=1, D6=0, D7=1

To find parity bit p1 value, take data bits D3, D5, D7=1, 1, 1 apply even parity , then p1 =1

To find parity bit p2 value, take data bits D3, D6, D7=1, 0, 1 apply even parity , then p2=0

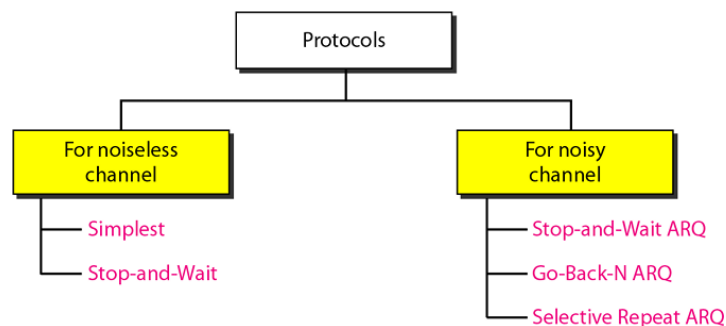
To find parity bit p4 value, take data bits D5, D6, D7=1, 0, 1 apply even parity , then p4=0

After finding parity bit hamming code = 1010101

Elementary Data Link Layer protocols:

Now let us see how the data link layer can combine framing, flow control, and error control to achieve the delivery of data from one node to another. The protocols are normally implemented in software by using one of the common programming languages.

We divide the discussion of protocols into those that can be used for noiseless (error-free) channels and those that can be used for noisy (error-creating) channels. The protocols in the first category cannot be used in real life, but they serve as a basis for understanding the protocols of noisy channels.



NOISELESS CHANNELS:

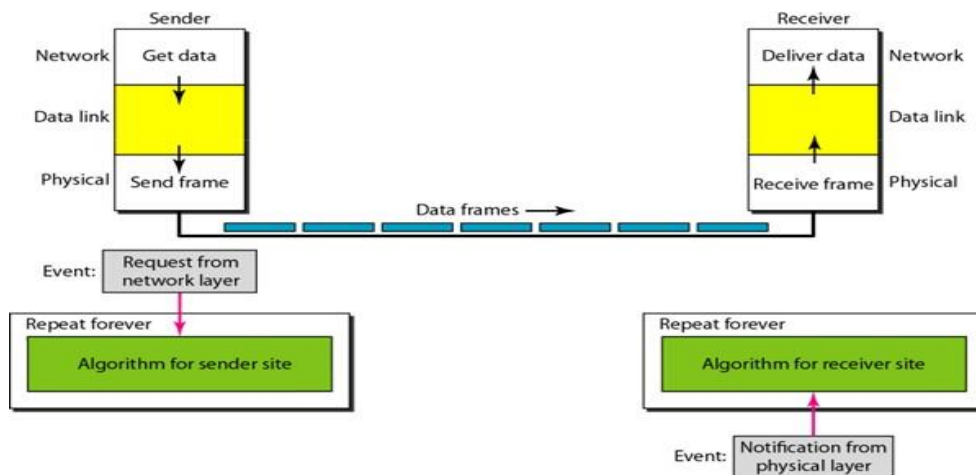
Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel. The first is a protocol that does not use flow control; the second is the one that does. Of course, neither has error control because we have assumed that the channel is a perfect noiseless channel.

Simplest Protocol:

Our first protocol, which we call the Simplest Protocol for lack of any other name, is one that has no flow or error control. Like other protocols we will discuss in this chapter, it is a unidirectional protocol in which data frames are traveling in only one direction—from the sender to receiver. We assume that the receiver can immediately handle any frame it receives with a processing time that is small enough to be negligible. The data link layer of the receiver immediately removes the header from the frame and hands the data packet to its network layer, which can also accept the packet immediately. In other words, the receiver can never be overwhelmed with incoming frames.

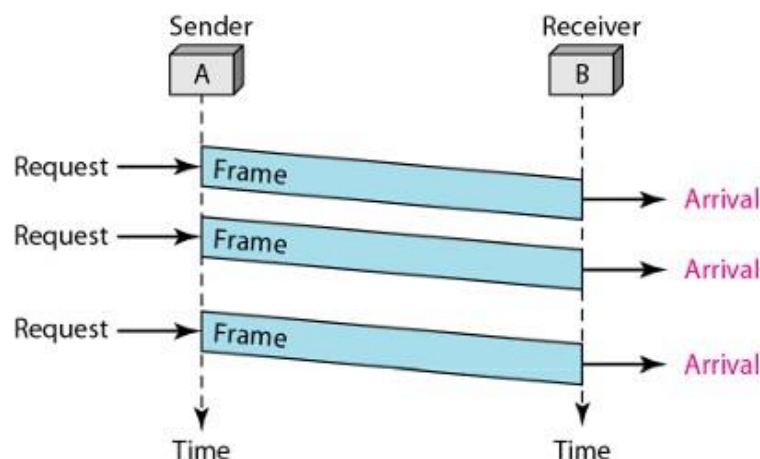
Design

There is no need for flow control in this scheme. The data link layer at the sender site gets data from its network layer, makes a frame out of the data, and sends it. The data link layer at the receiver site receives a frame from its physical layer, extracts data from the frame, and delivers the data to its network layer. The data link layers of the sender and receiver provide transmission services for their network layers.



Example:

Below Figure shows an example of communication using this protocol. It is very simple. The sender sends a sequence of frames without even thinking about the receiver. To send three frames, three events occur at the sender site and three events at the receiver site. Note that the data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.



Elementary Data Link Layer protocols: Simplex protocol, Simplex stop and wait, Simplex protocol for Noisy Channel. Sliding window protocol: One bit, Go-back N, Selective Repetitive protocol, Stop and wait protocol

Stop and Wait Protocol

Before understanding the stop and Wait protocol, we first know about the error control mechanism. The error control mechanism is used so that the received data should be exactly same whatever sender has sent the data. The error control mechanism is divided into two categories, i.e., Stop and Wait ARQ and sliding window. The sliding window is further divided into two categories, i.e., Go Back N, and Selective Repeat. Based on the usage, the people select the error control mechanism whether it is **stop and wait** or **sliding window**.

Here stop and wait means, whatever the data that sender wants to send, he sends the data to the receiver. After sending the data, he stops and waits until he receives the acknowledgment from the receiver. The stop and wait protocol is a flow control protocol where flow control is one of the services of the data link layer.

It is a data-link layer protocol which is used for transmitting the data over the noiseless channels. It provides unidirectional data transmission which means that either sending or receiving of data will take place at a time. It provides flow-control mechanism but does not provide any error control mechanism.

The idea behind the usage of this frame is that when the sender sends the frame then he waits for the acknowledgment before sending the next frame.

Primitives of Stop and Wait Protocol

The primitives of stop and wait protocol are:

Sender side

Rule 1: Sender sends one data packet at a time.

Rule 2: Sender sends the next packet only when it receives the acknowledgment of the previous packet.

Therefore, the idea of stop and wait protocol in the sender's side is very simple, i.e., send one packet at a time, and do not send another packet before receiving the acknowledgment.

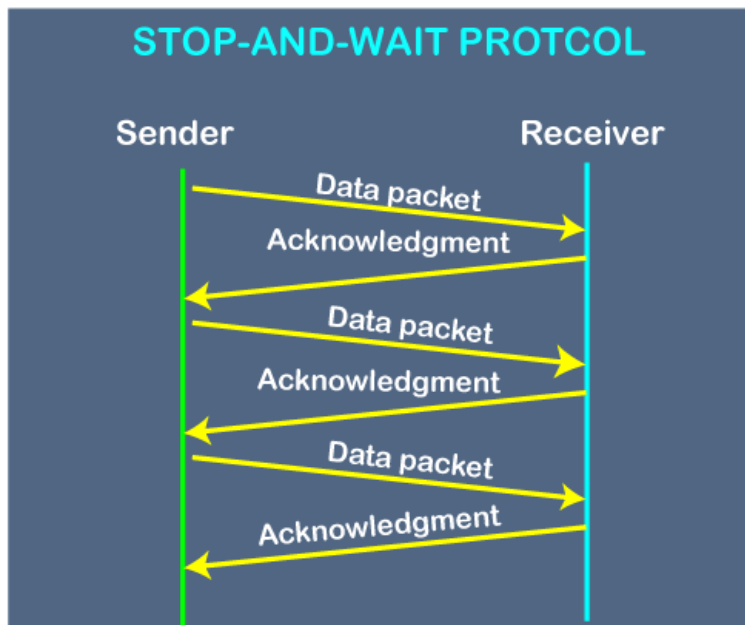
Receiver side

Rule 1: Receive and then consume the data packet.

Rule 2: When the data packet is consumed, receiver sends the acknowledgment to the sender.

Therefore, the idea of stop and wait protocol in the receiver's side is also very simple, i.e., consume the packet, and once the packet is consumed, the acknowledgment is sent. This is known as a flow control mechanism.

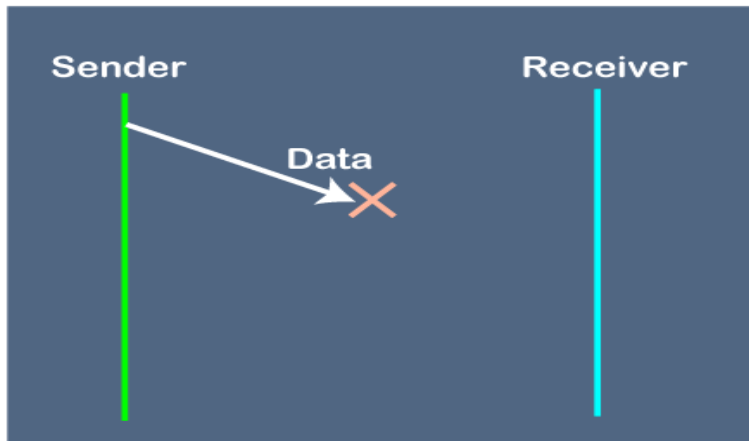
Working of Stop and Wait protocol



The above figure shows the working of the stop and wait protocol. If there is a sender and receiver, then sender sends the packet and that packet is known as a data packet. The sender will not send the second packet without receiving the acknowledgment of the first packet. The receiver sends the acknowledgment for the data packet that it has received. Once the acknowledgment is received, the sender sends the next packet. This process continues until all the packet are not sent. The main advantage of this protocol is its simplicity but it has some disadvantages also. For example, if there are 1000 data packets to be sent, then all the 1000 packets cannot be sent at a time as in Stop and Wait protocol, one packet is sent at a time.

Disadvantages of Stop and Wait protocol

The following are the problems associated with a stop and wait protocol: 1. Problems occur due to lost data

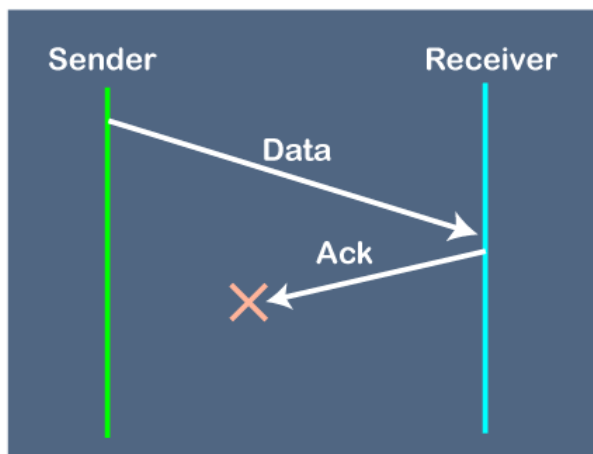


Suppose the sender sends the data and the data is lost. The receiver is waiting for the data for a long time. Since the data is not received by the receiver, so it does not send any acknowledgment. Since the sender does not receive any acknowledgment so it will not send the next packet. This problem occurs due to the lost data.

In this case, two problems occur:

- Sender waits for an infinite amount of time for an acknowledgment.
- Receiver waits for an infinite amount of time for a data.

2. Problems occur due to lost acknowledgment

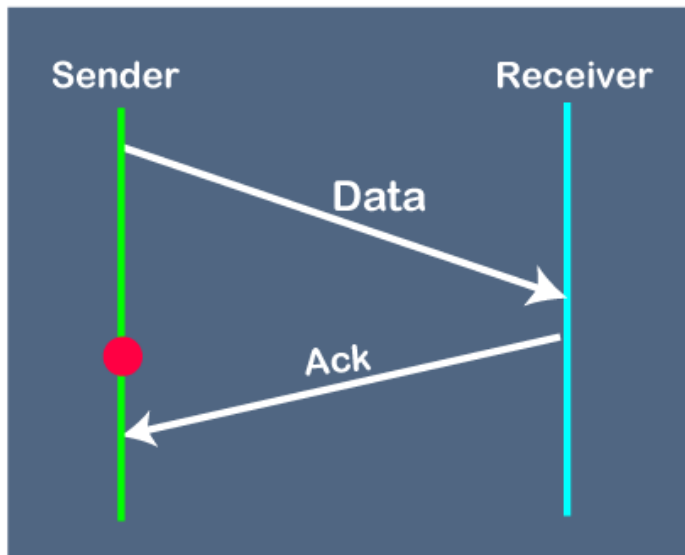


Suppose the sender sends the data and it has also been received by the receiver. On receiving the packet, the receiver sends the acknowledgment. In this case, the acknowledgment is lost in a network, so there is no chance for the sender to receive the acknowledgment. There is also no chance for the sender to send the next packet as in stop and wait protocol, the next packet cannot be sent until the acknowledgment of the previous packet is received.

In this case, one problem occurs:

- Sender waits for an infinite amount of time for an acknowledgment.

3. Problem due to the delayed data or acknowledgment



Suppose the sender sends the data and it has also been received by the receiver. The receiver then sends the acknowledgment but the acknowledgment is received after the timeout period on the sender's side. As the acknowledgment is received late, so acknowledgment can be wrongly considered as the acknowledgment of some other data packet.

Stop and Wait

Characteristics

- Used in Connection-oriented communication.
- It offers error and flows control
- It is used in Data Link and Transport Layers
- Stop and Wait for ARQ mainly implements the Sliding Window Protocol concept with Window Size 1

Sender:

Rule 1) Send one data packet at a time.

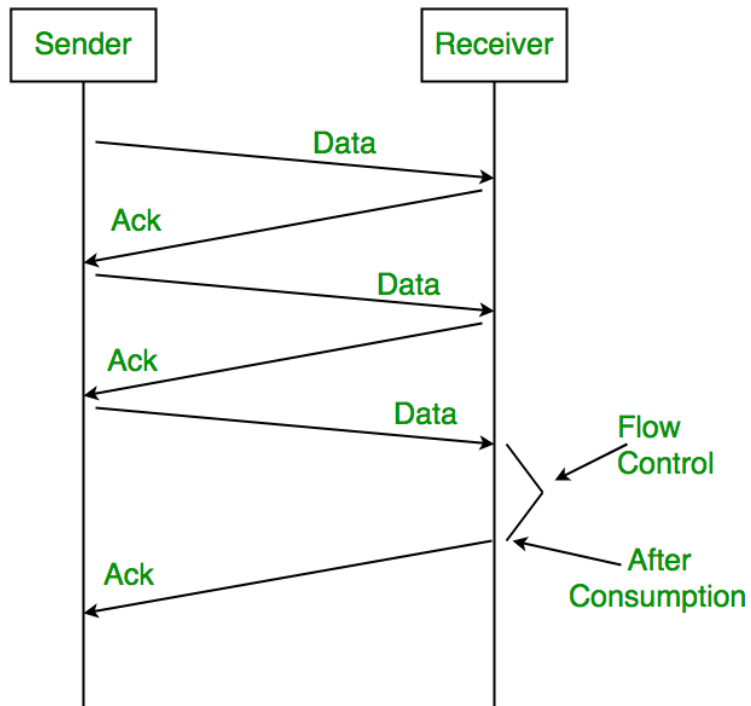
Rule 2) Send the next packet only after receiving acknowledgement for the previous.

Receiver:

Rule 1) Send acknowledgement after receiving and consuming a data packet.

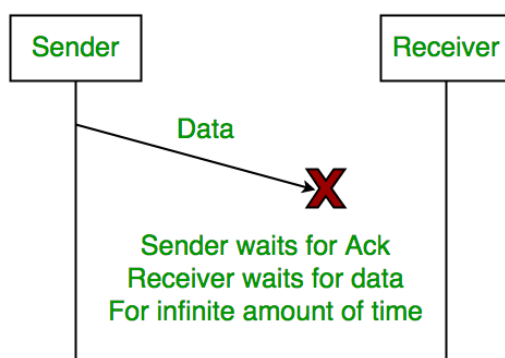
Rule 2) After consuming packet acknowledgement need to be sent (Flow Control)

Stop and Wait Data flow Diagram:

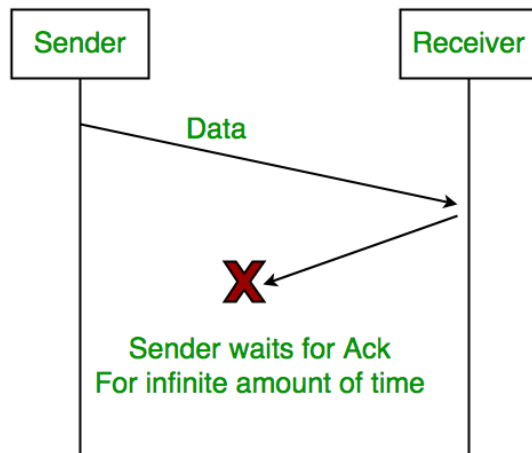


Problems :

1. Lost Data



2. Lost Acknowledgement:

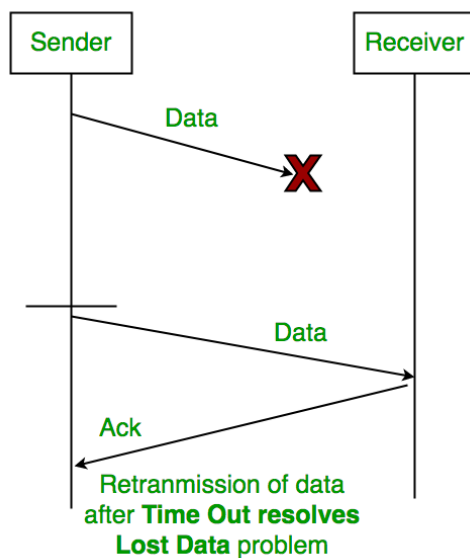


3. Delayed Acknowledgement/Data: After a timeout on the sender side, a long-delayed acknowledgement might be wrongly considered as acknowledgement of some other recent packet.

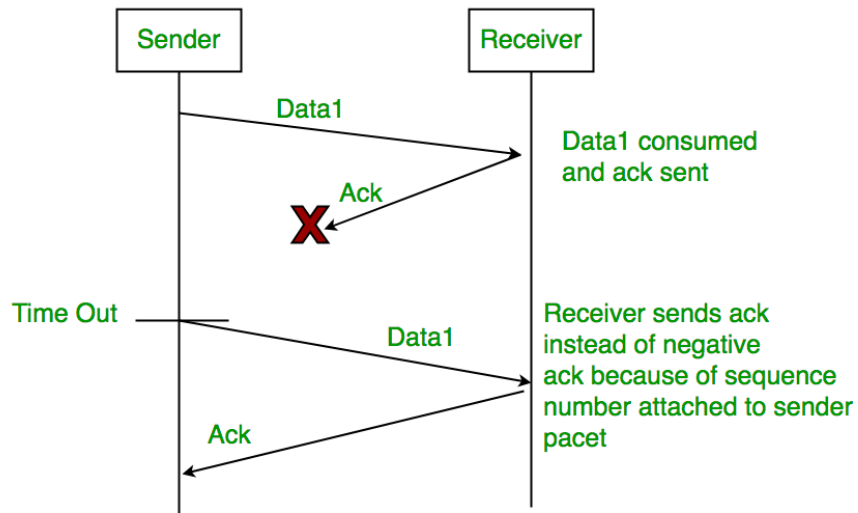
Stop and Wait ARQ (Automatic Repeat Request)

The above 3 problems are resolved by Stop and Wait for ARQ (Automatic Repeat Request) that does both error control and flow control.

1. Time out:



2. Sequence Number (Data)

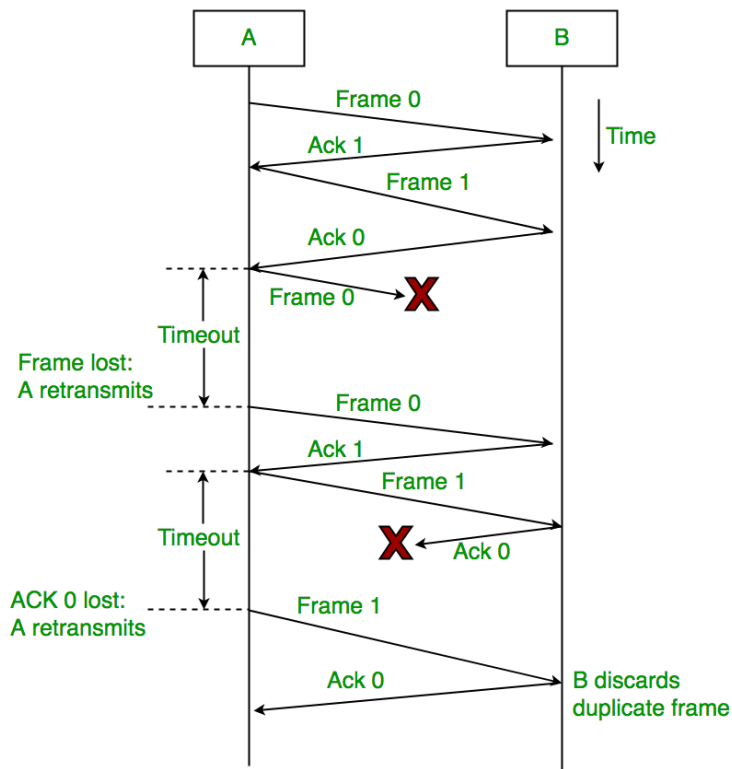


3. Delayed Acknowledgement:

This is resolved by introducing sequence numbers for acknowledgement also.

Working of Stop and Wait ARQ:

- 1) Sender A sends a data frame or packet with sequence number 0.
 - 2) Receiver B, after receiving the data frame, sends an acknowledgement with sequence number 1 (the sequence number of the next expected data frame or packet)
- There is only a one-bit sequence number that implies that both sender and receiver have a buffer for one frame or packet only.



Characteristics of Stop and Wait ARQ:

- It uses a link between sender and receiver as a half-duplex link
- The sender has to keep waiting for acknowledgements before sending the processed next packet.
- It is a special category of SWP where its window size is 1
- Irrespective of the number of packets sender is having stop and wait for protocol requires only 2 sequence numbers 0 and 1

Constraints:

Stop and Wait ARQ has very less efficiency, it can be improved by increasing the window size. Also, for better efficiency, Go back N and Selective Repeat Protocols are used.

The Stop and Wait ARQ solves the main three problems but may cause big performance issues as the sender always waits for acknowledgement even if it has the next packet ready to send. Consider a situation where you have a high bandwidth connection and propagation delay is also high (you are connected to some server in some other country through a high-speed connection). To solve this problem, we can send more than one packet at a time with a larger sequence number. We will be discussing these protocols in the next articles.

So Stop and Wait ARQ may work fine where propagation delay is very less for example LAN connections but performs badly for distant connections like satellite connections.

Advantages of Stop and Wait ARQ :

- **Simple Implementation:** Stop and Wait ARQ is a simple protocol that is easy to implement in both hardware and software. It does not require complex algorithms or hardware components, making it an inexpensive and efficient option.
- **Error Detection:** Stop and Wait ARQ detects errors in the transmitted data by using checksums or cyclic redundancy checks (CRC). If an error is detected, the receiver sends a negative acknowledgment (NAK) to the sender, indicating that the data needs to be retransmitted.
- **Reliable:** Stop and Wait ARQ ensures that the data is transmitted reliably and in order. The receiver cannot move on to the next data packet until it receives the current one. This ensures that the data is received in the correct order and eliminates the possibility of data corruption.
- **Flow Control:** Stop and Wait ARQ can be used for flow control, where the receiver can control the rate at which the sender transmits data. This is useful in situations where the receiver has limited buffer space or processing power.
- **Backward Compatibility:** Stop and Wait ARQ is compatible with many existing systems and protocols, making it a popular choice for communication over unreliable channels.

Disadvantages of Stop and Wait ARQ :

- **Low Efficiency:** Stop and Wait ARQ has low efficiency as it requires the sender to wait for an acknowledgment from the receiver before sending the next data packet. This results in a low data transmission rate, especially for large data sets.
- **High Latency:** Stop and Wait ARQ introduces additional latency in the transmission of data, as the sender must wait for an acknowledgment before sending the next packet. This can be a problem for real-time applications such as video streaming or online gaming.
- **Limited Bandwidth Utilization:** Stop and Wait ARQ does not utilize the available bandwidth efficiently, as the sender can transmit only one data packet at a time. This results in underutilization of the channel, which can be a problem in situations where the available bandwidth is limited.
- **Limited Error Recovery:** Stop and Wait ARQ has limited error recovery capabilities. If a data packet is lost or corrupted, the sender must retransmit the entire packet, which can be time-consuming and can result in further delays.
- **Vulnerable to Channel Noise:** Stop and Wait ARQ is vulnerable to channel noise, which can cause errors in the transmitted data. This can result in frequent retransmissions and can impact the overall efficiency of the protocol.

Sliding Window Protocol

The sliding window is a technique for sending multiple frames at a time. It controls the data packets between the two devices where reliable and gradual delivery of data frames is needed. It is also used in TCP (Transmission Control Protocol).

In this technique, each frame has sent from the sequence number. The sequence numbers are used to find the missing data in the receiver end. The purpose of the sliding window technique is to avoid duplicate data, so it uses the sequence number.

Types of Sliding Window Protocol

Sliding window protocol has two types:

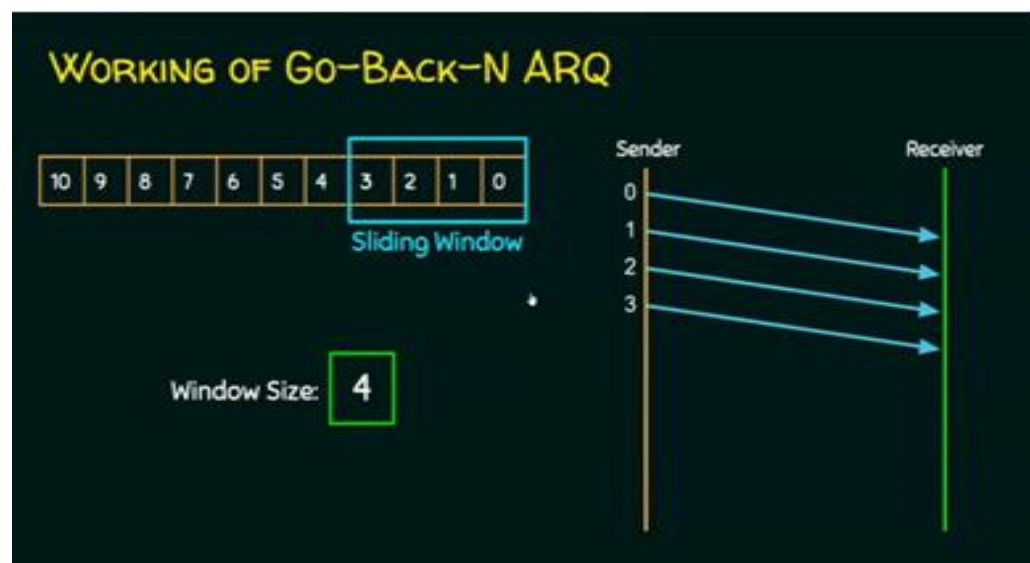
1. Go-Back-N ARQ
2. Selective Repeat ARQ

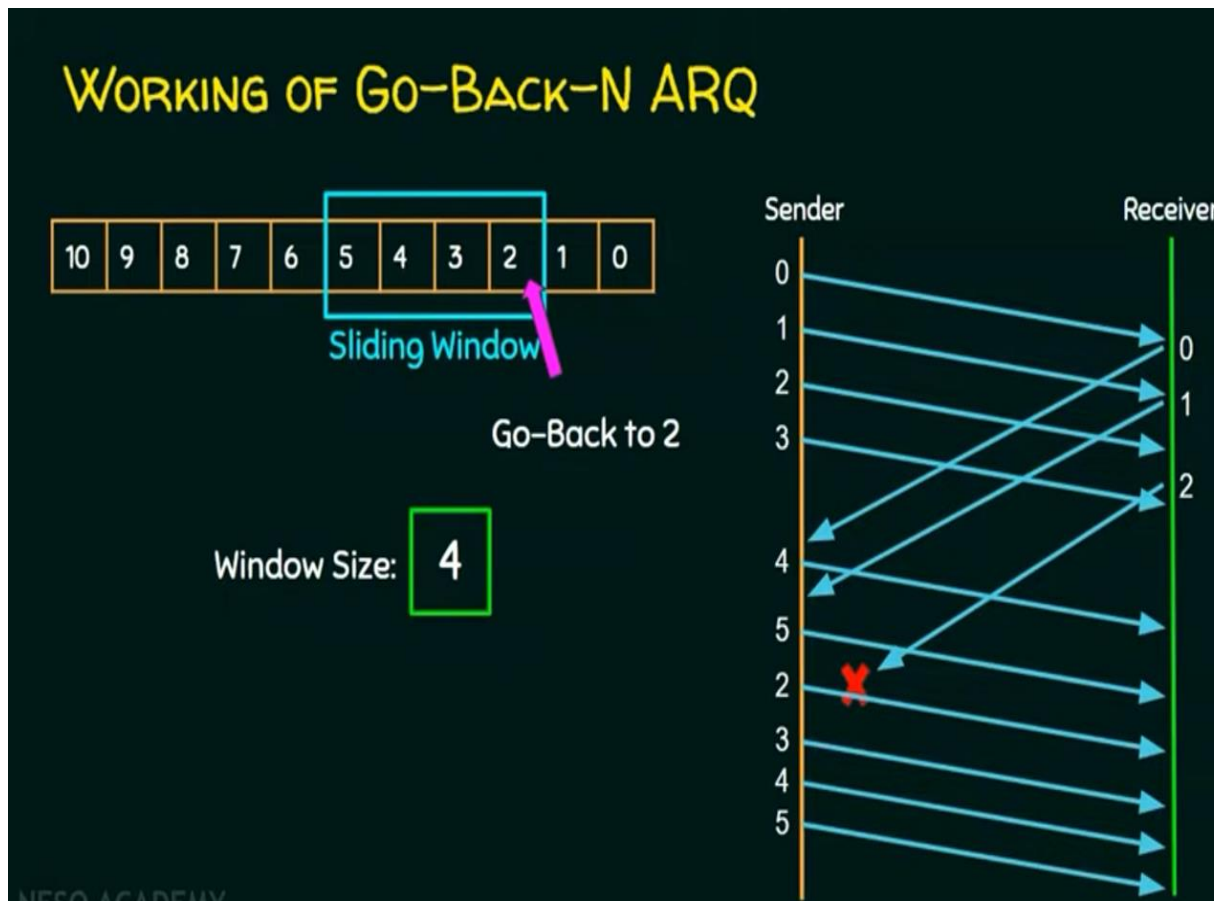
Go-Back-N ARQ

Go-Back-N ARQ protocol is also known as Go-Back-N Automatic Repeat Request. It is a data link layer protocol that uses a sliding window method. In this, if any frame is corrupted or lost, all subsequent frames have to be sent again.

The size of the sender window is N in this protocol. For example, Go-Back-8, the size of the sender window, will be 8. The receiver window size is always 1.

If the receiver receives a corrupted frame, it cancels it. The receiver does not accept a corrupted frame. When the timer expires, the sender sends the correct frame again. The design of the Go-Back-N ARQ protocol is shown below.



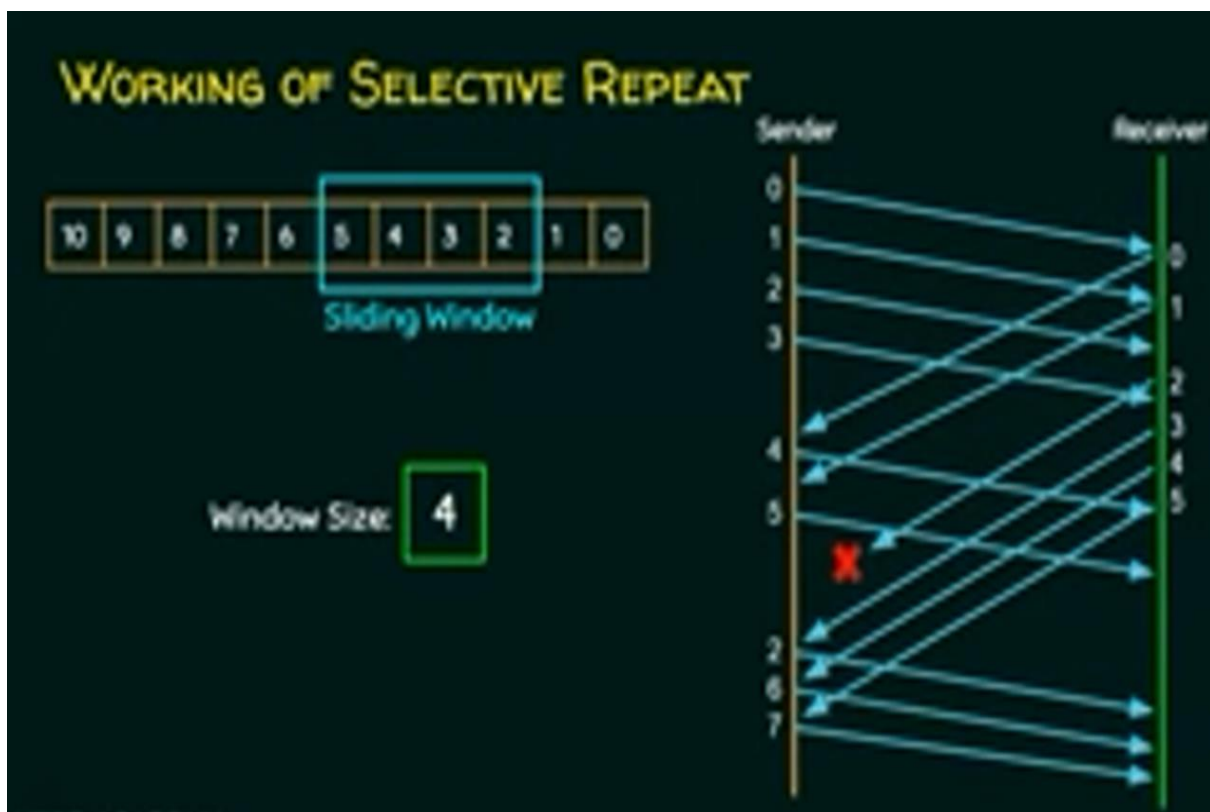
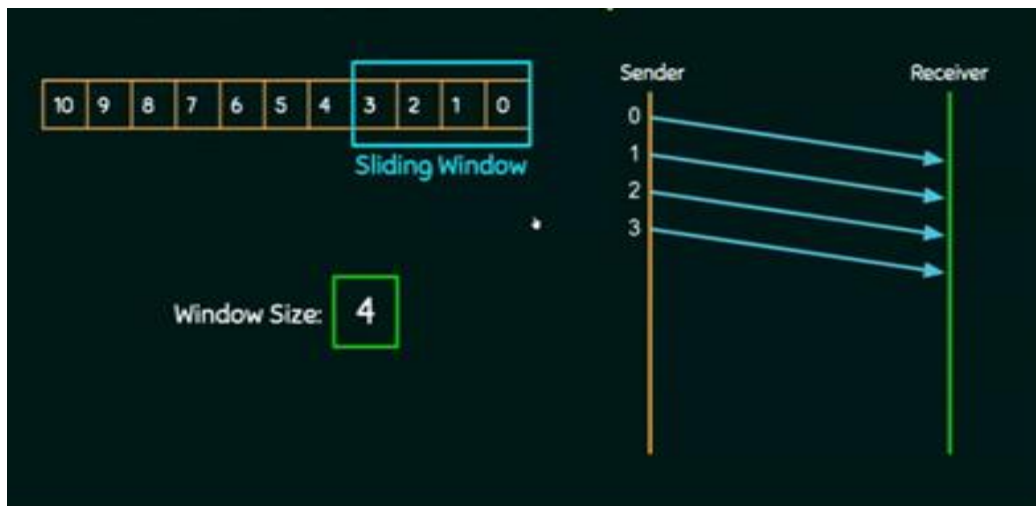


The example of Go-Back-N ARQ is shown above in the figure.

Selective Repeat ARQ

Selective Repeat ARQ is also known as the Selective Repeat Automatic Repeat Request. It is a data link layer protocol that uses a sliding window method. The Go-back-N ARQ protocol works well if it has fewer errors. But if there is a lot of error in the frame, lots of bandwidth loss in sending the frames again. So, we use the Selective Repeat ARQ protocol. In this protocol, the size of the sender window is always equal to the size of the receiver window. The size of the sliding window is always greater than 1.

If the receiver receives a corrupt frame, it does not directly discard it. It sends a negative acknowledgment to the sender. The sender sends that frame again as soon as on the receiving negative acknowledgment. There is no waiting for any time-out to send that frame. The design of the Selective Repeat ARQ protocol is shown below.



The example of the Selective Repeat ARQ protocol is shown above in the figure.

Differences between the Go-Back-N ARQ and Selective Repeat ARQ

Go-Back-N ARQ

If a frame is corrupted or lost in it, all subsequent frames have to be sent again.

If it has a high error rate, it wastes a lot

Selective Repeat ARQ

In this, only the frame is sent again, which is corrupted or lost.

There is a loss of low bandwidth.

of bandwidth.

It is less complex.

It does not require sorting.

It does not require searching.

It is used more.

It is more complex because it has to do sorting and searching as well. And it also requires more storage.

In this, sorting is done to get the frames in the correct order.

The search operation is performed in it.

It is used less because it is more complex.

HDLC:

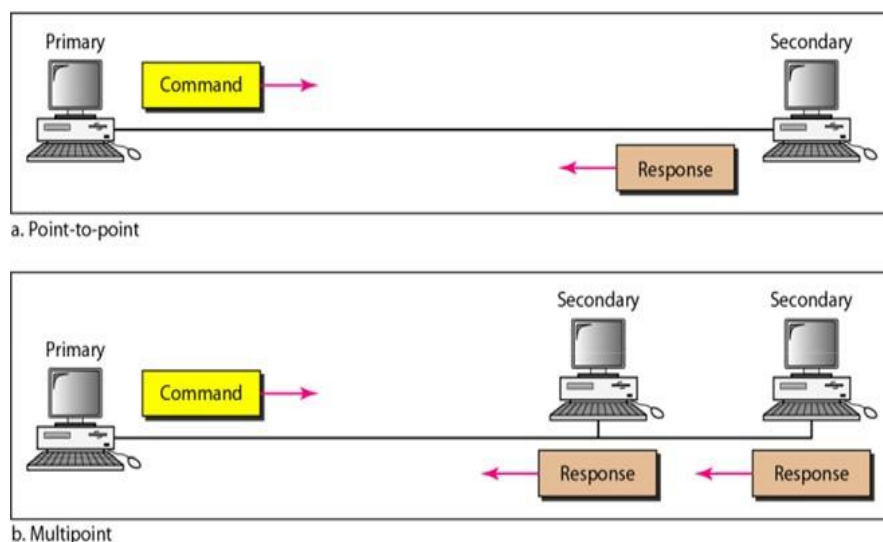
High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. It implements the ARQ mechanisms

Configurations and Transfer Modes

HDLC provides two common transfer modes that can be used in different configurations: normal response mode (NRM) and asynchronous balanced mode (ABM).

Normal Response Mode

In normal response mode (NRM), the station configuration is unbalanced. We have one primary station and multiple secondary stations. A primary station can send commands; a secondary station can only respond. The NRM is used for both point-to-point and multiple-point links, as shown in below Figure.



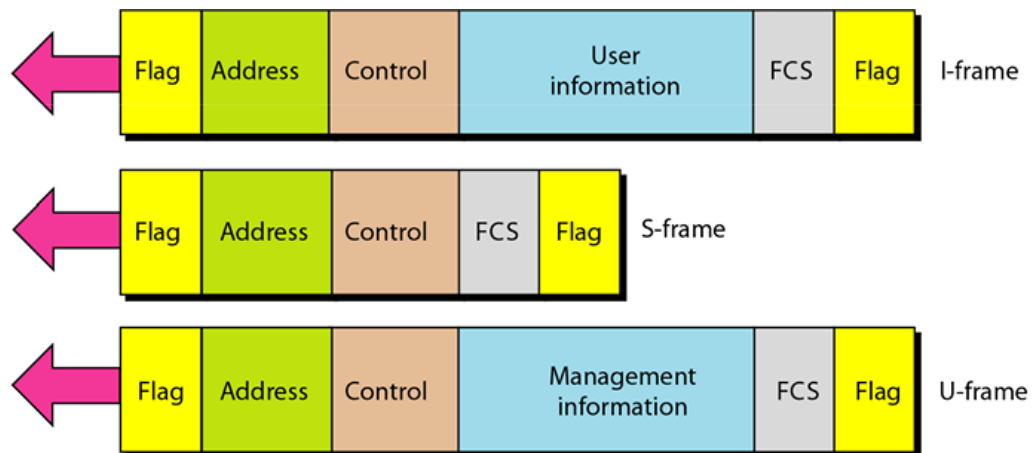
Asynchronous Balanced Mode

In asynchronous balanced mode (ABM), the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in below Figure. This is the common mode today.



Frames

To provide the flexibility necessary to support all the options possible in the modes and configurations just described, HDLC defines three types of frames: information frames (I-frames), supervisory frames (S-frames), and unnumbered frames (V-frames). Each type of frame serves as an envelope for the transmission of a different type of message. I-frames are used to transport user data and control information relating to user data (piggybacking). S-frames are used only to transport control information. V-frames are reserved for system management. Information carried by V-frames is intended for managing the link itself.



Fields

Let us now discuss the fields and their use in different frame types.

Flag field. The flag field of an HDLC frame is an 8-bit sequence with the bit pattern 01111110 that identifies both the beginning and the end of a frame and serves as a synchronization pattern for the receiver.

Address field. The second field of an HDLC frame contains the address of the secondary station. If a primary station created the frame, it contains a *to* address. If a secondary creates the frame, it contains a *from* address. An address field can be 1 byte or several bytes long, depending on the needs of the network. One byte can identify up to 128 stations (1 bit is used for another purpose). Larger networks require multiple-byte address

fields. If the address field is only 1 byte, the last bit is always a 1. If the address is more than 1 byte, all bytes but the last one will end with 0; only the last will end with 1. Ending each intermediate byte with 0 indicates to the receiver that there are more address bytes to come.

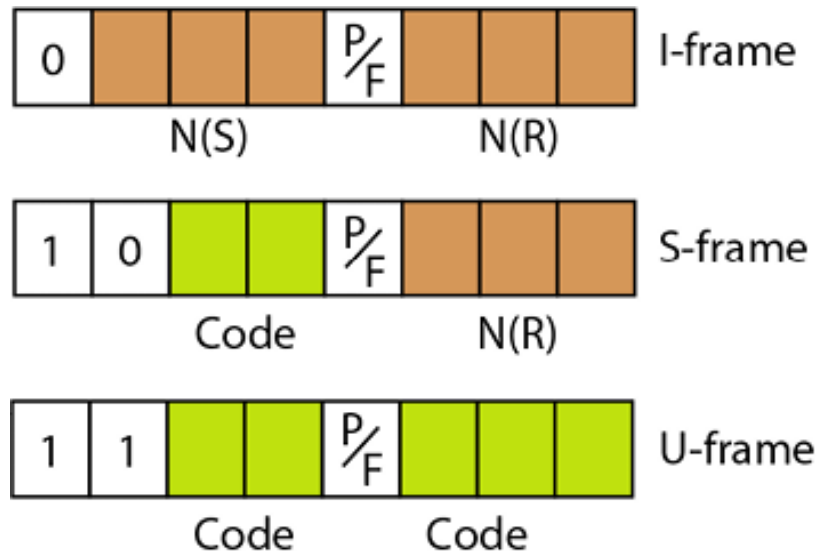
Control field. The control field is a 1- or 2-byte segment of the frame used for flow and error control. The interpretation of bits in this field depends on the frame type. We discuss this field later and describe its format for each frame type.

Information field. The information field contains the user's data from the network layer or management information. Its length can vary from one network to another.

FCS field. The frame check sequence (FCS) is the HDLC error detection field. It can contain either a 2- or 4- byte ITU-T CRC.

Control Field

The control field determines the type of frame and defines its functionality. So let us discuss the format of this field in greater detail. The format is specific for the type of frame, as shown in below Figure.



Control Field for I-Frames

I-frames are designed to carry user data from the network layer. In addition, they can include flow and error control information (piggybacking). The subfields in the control field are used to define these functions. The first bit defines the type. If the first bit of the control field is 0, this means the frame is an I-frame. The next 3 bits, called $N(S)$, define the sequence number of the frame. Note that with 3 bits, we can define a sequence number between 0 and 7; but in the extension format, in which the control field is 2 bytes, this field is larger. The last 3 bits, called $N(R)$, correspond to the acknowledgment number when piggybacking is used. The single bit between $N(S)$ and $N(R)$ is called the *PIF* bit. The *PIF* field is a single bit with a dual purpose. It has meaning only when it is set (bit = 1) and can mean *poll* or *final*. It means *poll* when the frame is sent by a primary station to a secondary (when the address field contains the address of the receiver). It means *final* when the frame is sent by a secondary to a primary (when the address field contains the address of the sender).

Control Field for S-Frames

Supervisory frames are used for flow and error control whenever piggybacking is either impossible or inappropriate (e.g., when the station either has no data of its own to send or needs to send a command or response other than an acknowledgment). S-frames do not have information fields. If the first 2 bits of the control field is 10, this means the frame is an S-frame. The last 3 bits, called $N(R)$, corresponds to the acknowledgment number (ACK) or negative acknowledgment number (NAK) depending on the type of S-frame. The 2 bits called **code** is used to define the type of S-frame itself. With 2 bits, we can have four types of S-frames, as described below:

Receive ready (RR). If the value of the code subfield is 00, it is an RR S-frame. This kind of frame acknowledges the receipt of a safe and sound frame or group of frames. In this case, the value $N(R)$ field defines the acknowledgment number.

Receive not ready (RNR). If the value of the code subfield is 10, it is an RNR S-frame. This kind of frame is an RR frame with additional functions. It acknowledges the receipt of a frame or group of frames, and it announces that the receiver is busy and cannot receive more frames. It acts as a kind of congestion control mechanism by asking the sender to slow down. The value of NCR is the acknowledgment number.

Reject (REJ). If the value of the code subfield is 01, it is a REJ S-frame. This is a NAK frame, but not like the one used for Selective Repeat ARQ. It is a NAK that can be used in Go-Back-N ARQ to improve the efficiency of the process by informing the sender, before the sender time expires, that the last frame is lost or damaged. The value of NCR is the negative acknowledgment number.

Selective reject (SREJ). If the value of the code subfield is 11, it is an SREJ S-frame. This is a NAK frame used in Selective Repeat ARQ. Note that the HDLC Protocol uses the term *selective reject* instead of *selective repeat*. The value of $N(R)$ is the negative acknowledgment number.

Control Field for U-Frames: Unnumbered frames are used to exchange session management and control information between connected devices. Unlike S-frames, U-frames contain an information field, but one used for system management information, not user data. As with S-frames, however, much of the information carried by U-frames is contained in codes included in the control field. U-frame codes are divided into two sections: a 2-bit prefix before the P/F bit and a 3-bit suffix after the P/F bit. Together, these two segments (5 bits) can be used to create up to 32 different types of U-frames. Some of the more common types are shown in below Table.

U-frame control command and response

<i>Code</i>	<i>Command</i>	<i>Response</i>	<i>Meaning</i>
00 001	SNRM		Set normal response mode
11 011	SNRME		Set normal response mode, extended
11 100	SABM	DM	Set asynchronous balanced mode or disconnect mode
11 110	SABME		Set asynchronous balanced mode, extended
00 000	UI	UI	Unnumbered information
00 110		UA	Unnumbered acknowledgment
00 010	DISC	RD	Disconnect or request disconnect
10 000	SIM	RIM	Set initialization mode or request information mode
00 100	UP		Unnumbered poll
11 001	RSET		Reset
11 101	XID	XID	Exchange ID
10 001	FRMR	FRMR	Frame reject

POINT-TO-POINT PROTOCOL:

Although HDLC is a general protocol that can be used for both point-to-point and multipoint configurations, one of the most common protocols for point-to-point access is the Point-to-Point Protocol (PPP). Today, millions of Internet users who need to connect their home computers to the server of an Internet service provider use PPP. The majority of these users have a traditional modem; they are connected to the Internet through a telephone line, which provides the services of the physical layer. But to control and manage the transfer of data, there is a need for a point-to-point protocol at the data link layer. PPP is by far the most common.

PPP provides several services:

1. PPP defines the format of the frame to be exchanged between devices.
2. PPP defines how two devices can negotiate the establishment of the link and the exchange of data.
3. PPP defines how network layer data are encapsulated in the data link frame.
4. PPP defines how two devices can authenticate each other.
5. PPP provides multiple network layer services supporting a variety of network layer protocols.
6. PPP provides connections over multiple links.
7. PPP provides network address configuration. This is particularly useful when a home user needs a temporary network address to connect to the Internet.

On the other hand, to keep PPP simple, several services are missing:

I. PPP does not provide flow control. A sender can send several frames one after another with no concern about overwhelming the receiver.

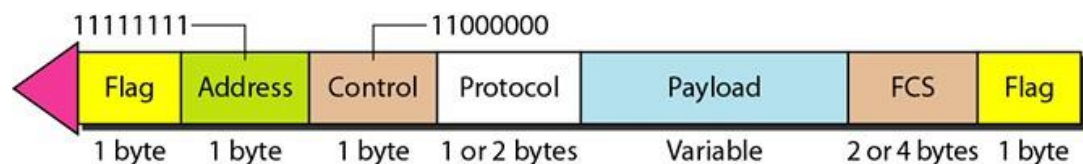
1. PPP has a very simple mechanism for error control. A CRC field is used to detect errors. If the frame is corrupted, it is silently discarded; the upper-layer protocol needs to take care of the problem. Lack of error control and sequence numbering may cause a packet to be received out of order.
2. PPP does not provide a sophisticated addressing mechanism to handle frames in a multipoint configuration.

Framing:

PPP is a byte-oriented protocol. Framing is done according to the discussion of byte oriented protocols.

Frame Format

Below Figure shows the format of a PPP frame. The description of each field follows:



Flag. A PPP frame starts and ends with a 1-byte flag with the bit pattern 01111110. Although this pattern is the same as that used in HDLC, there is a big difference. PPP is a byte-oriented protocol; HDLC is a bit-oriented protocol. The flag is treated as a byte, as we will explain later.

Address. The address field in this protocol is a constant value and set to 11111111(broadcast address). During negotiation (discussed later), the two parties may agree to omit this byte.

Control. This field is set to the constant value 11000000 (imitating unnumbered frames in HDLC). As we will discuss later, PPP does not provide any flow control. Error control is also limited to error detection. This means that this field is not needed at all, and again, the two parties can agree, during negotiation, to omit this byte.

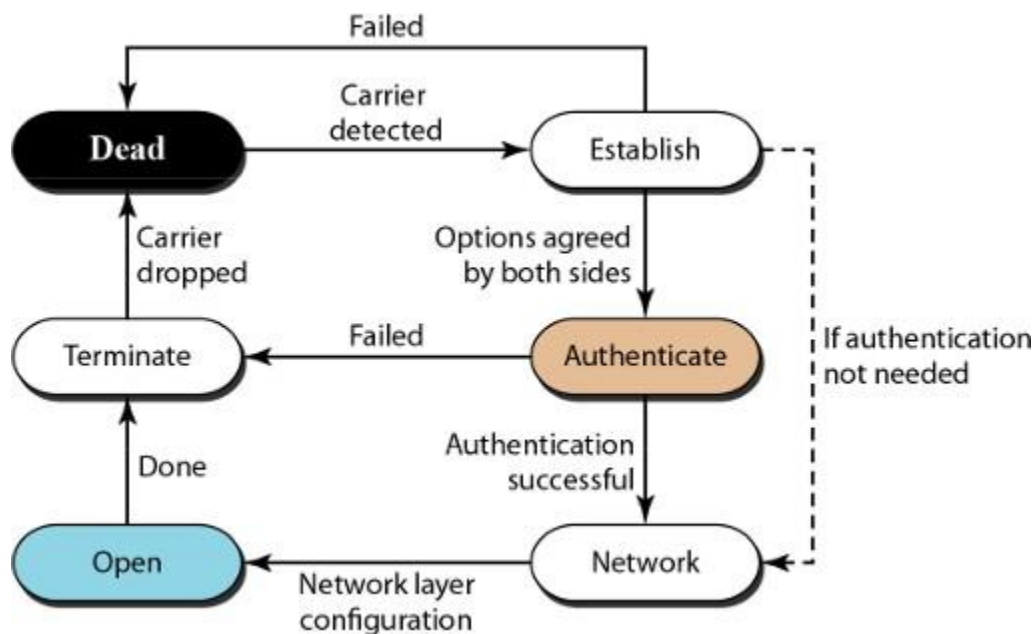
Protocol. The protocol field defines what is being carried in the data field: either user data or other information. We discuss this field in detail shortly. This field is by default 2 bytes long, but the two parties can agree to use only 1 byte.

Payload field. This field carries either the user data or other information that we will discuss shortly. The data field is a sequence of bytes with the default of a maximum of 1500 bytes; but this can be changed during negotiation. The data field is byte stuffed if the flag byte pattern appears in this field. Because there is no field defining the size of the data field, padding is needed if the size is less than the maximum default value or the maximum negotiated value.

FCS. The frame check sequence (FCS) is simply a 2-byte or 4-byte standard CRC.

Transition Phases

A PPP connection goes through phases which can be shown in a transition phase diagram.



Dead. In the dead phase the link is not being used. There is no active carrier (at the physical layer) and the line is quiet.

Establish. When one of the nodes starts the communication, the connection goes into this phase. In this phase, options are negotiated between the two parties. If the negotiation is successful, the system goes to the authentication phase (if authentication is required) or directly to the networking phase. The link control protocol packets, discussed shortly, are used for this purpose. Several packets may be exchanged here.

Authenticate. The authentication phase is optional; the two nodes may decide, during the establishment phase, not to skip this phase. However, if they decide to proceed with

authentication, they send several authentication packets, discussed later. If the result is successful, the connection goes to the networking phase; otherwise, it goes to the termination phase.

Network. In the network phase, negotiation for the network layer protocols takes place. PPP specifies that two nodes establish a network layer agreement before data at the network layer can be exchanged. The reason is that PPP supports multiple protocols at the network layer. If a node is running multiple protocols simultaneously at the network layer, the receiving node needs to know which protocol will receive the data.

Open. In the open phase, data transfer takes place. When a connection reaches this phase, the exchange of data packets can be started. The connection remains in this phase until one of the endpoints wants to terminate the connection.

Terminate. In the termination phase the connection is terminated. Several packets are exchanged between the two ends for house cleaning and closing the link.